A Report

on

FlowCSV

carried out as part of the course Project Based Learning- 4

Submitted by

Progyan Deb

23FE10CSE00197

VI Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Department of Computer Science & Engineering,
School of Computer Science and Engineering,
Manipal University Jaipur,
Jan-May 2026

Acknowledgement

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor ***Dr. Ajay Kumar*** for his/her constant support, guidance, and continuous engagement. I highly appreciate his technical comments, suggestions, and criticism during the progress of this project titles "**FlowCSV**".

I owe my profound gratitude to ***Dr. Neha Chaudhary*** , Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Progyan Deb

23FE10CSE00197

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: _____

CERTIFICATE

This is to certify that the project entitled "***FlowCSV***" is a bonafide work carried out as PBL-4 ***End Term Assessment*** in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by ***Progyan Deb*** bearing registration number 23FE10CSE00197, during the academic semester VI *of year 2025-2026.*

Place: Manipal University Jaipur, Jaipur

Name of the project guide: Dr. Ajay Kumar

Signature of the project guide: _____

Table of Contents

1. Introduction *6*

1.1 Objective of the Project **6**

1.2 Brief Description of the Project **6**

1.3 Technology Used **6**

1.3.1 Hardware Requirements 6

1.3.2 Software Requirements 7

1.4 Organization Profile **7**

2. Design Description *8*

2.1 Flow Chart **8**

2.2 Data Flow Diagrams (DFDs) **9**

Level 0 DFD — Context Diagram 9

Level 1 DFD — System Processes 9

2.3 Entity Relationship Diagram (E-R Diagram) **10**

3. Project Description *12*

3.1 Database **12**

3.2 Table Description **12**

Job Object Schema 12

Pipeline Step Object Schema 13

3.3 File / Database Design **13**

4. Input / Output Form Design *14*

4.1 Input Design **14**

Upload Page — Input Fields 14

Input Data Format — CSV 14

Input Data Format — JSON 14

4.2 Output Design **14**

Dashboard Output Elements 15

Output File Format 15

5. Testing & Tools Used *16*

5.1 Testing Approach **16**

Functional Testing 16

Boundary Testing 16

5.2 Tools Used for Testing **17**

6. Implementation & Maintenance *18*

6.1 Implementation Details **18**

Phase 1 — Backend Foundation 18

Phase 2 — Pipeline Development 18

Phase 3 — Frontend Development 18

Phase 4 — Integration & Testing 18

6.2 Deployment Instructions **18**

6.3 Maintenance Guidelines **18**

7. Conclusion and Future Work ***20***

7.1 Conclusion **20**

7.2 Future Work **20**

Version 2 — Near-term Improvements 20

Version 3 — Scale Improvements 20

Version 4 — Intelligence Improvements 21

8. Outcome ***22***

8.1 Project Deployment **22**

8.2 Potential for Copyright / Open Source Publication **22**

9. Bibliography ***23***

1. Introduction

1.1 Objective of the Project

The primary objective of FlowCSV is to automate the end-to-end data preparation workflow that organizations, researchers, and analysts perform manually on a recurring basis. Specifically, the project aims to:

- Eliminate manual effort involved in reading, cleaning, and organizing CSV and JSON data files.
- Automatically detect and remove duplicate records from multi-source datasets.
- Categorize and split records into organized output files based on auto-detected or user-relevant fields.
- Provide a real-time visual dashboard that tracks each processing step as it executes.
- Generate a complete audit report for every pipeline run to support reproducibility and traceability.
- Reduce a 4–6 hour manual data preparation task to under 30 seconds with zero technical skill required from the end user.

1.2 Brief Description of the Project

FlowCSV is a full-stack web application that automates the data cleaning and organization pipeline for CSV and JSON files. Users upload one or more data files through a drag-and-drop interface. The system immediately processes those files through a 7-step automated pipeline: Initialize, Read, Clean, Organize, Generate, Archive, and Report.

Each step is executed asynchronously on the backend while the frontend polls for live status updates every 800 milliseconds, displaying real-time progress, per-step logs, and running statistics. Upon completion, users receive downloadable CSV files organized by category along with a JSON audit report summarizing all transformations performed.

The application is built on a React frontend communicating with an Express.js REST API backend. The pipeline logic runs entirely in Node.js, using csv-parse for parsing and csv-stringify for output generation. Job state is managed in-memory using a JavaScript Map, and processed files are automatically archived to prevent re-processing.

1.3 Technology Used

1.3.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent (2.0 GHz+)

Component	Minimum Specification
RAM	4 GB minimum; 8 GB recommended
Storage	500 MB free disk space for application + output files
Network	Local network or internet (for Google Fonts loading)
Display	1280 x 720 minimum resolution

1.3.2 Software Requirements

Software / Library	Version	Purpose
Node.js	v18.x or above	Backend runtime environment
React	v18.2.0	Frontend UI framework
Express.js	v4.18.2	REST API server
Multer	v1.4.5-lts	File upload middleware
csv-parse	v5.5.3	CSV file parsing
csv-stringify	v6.4.6	CSV file generation
react-dropzone	v14.2.3	Drag-and-drop file upload UI
uuid	v9.0.0	Unique job ID generation
concurrently	v8.2.2	Run frontend + backend together
VS Code / Any IDE	Latest	Code editor
npm	v9.x or above	Package manager
Web Browser	Chrome / Firefox (Latest)	Application access

1.4 Organization Profile

This project was developed as an academic submission for the Department of Computer Science & Engineering at Manipal University Jaipur. It is not affiliated with any external organization. The project was conceptualized and built independently as part of the final year project curriculum, addressing a real-world data processing problem that is universally applicable across industries including research, healthcare, finance, human resources, and operations management.

2. Design Description

2.1 Flow Chart

The following flowchart illustrates the complete user journey and processing flow within FlowCSV, from file upload through pipeline execution to output download.

START — User opens FlowCSV in browser
↓
User drags & drops CSV / JSON files onto Upload Page
↓ *File type validated (.csv / .json only)*
POST /api/upload → Multer saves files → Job created with UUID
↓ *Pipeline runs asynchronously in background*
Step 1 — Initialize: Create output & archive directories
↓
Step 2 — Read: Parse all CSV and JSON files into unified records
↓
Step 3 — Clean: Remove duplicate rows and empty records
↓
Step 4 — Organize: Group records by auto-detected category column
↓
Step 5 — Generate: Write one CSV file per category to disk
↓
Step 6 — Archive: Move source files to timestamped archive folder
↓
Step 7 — Report: Calculate metrics, write _report.json

↓ *Frontend polls GET /api/jobs/:id every 800ms*
Dashboard shows live step status, stats, output files
↓
END — User downloads organized CSV files + audit report

2.2 Data Flow Diagrams (DFDs)

Level 0 DFD — Context Diagram

The Level 0 DFD shows the system as a single process interacting with two external entities: the User (who uploads files and receives outputs) and the File System (where uploads, outputs, and archives are stored).

USER	**FlowCSV System**	**FILE SYSTEM**
(External Entity)	*Process 0: Automated Data Pipeline* ← Upload CSV/JSON files → Return cleaned organized files + report	*(External Entity)*

Level 1 DFD — System Processes

The Level 1 DFD decomposes the system into its core processes and the data flows between them.

Data Source	Process	Data Destination
User	P1 — Upload Handler: Receive files, validate format, save to /uploads, create Job ID	Job Store + File System
File System (/uploads)	P2 — Read: Parse CSV with csv-parse, parse JSON with JSON.parse, tag with source filename	In-memory record array
Record array	P3 — Clean: Filter empty rows, deduplicate using Set + JSON.stringify key	Clean record array
Clean records	P4 — Organize: Scan column names, detect category field, group records by value	Grouped record map
Grouped map	P5 — Generate: csv-stringify each group, write to /outputs/{jobId}/{category}.csv	File System (/outputs)

Data Source	Process	Data Destination
File System (/uploads)	P6 — Archive: Copy originals to /archives/{jobId}/, delete from /uploads	File System (/archives)
Job Store	P7 — Report: Compile metrics, write _report.json, finalize job status	File System + Job Store
Job Store	P8 — Polling: GET /api/jobs/:id returns current job state every 800ms	User (React frontend)

2.3 Entity Relationship Diagram (E-R Diagram)

Although FlowCSV uses in-memory storage (no persistent database), the following E-R diagram represents the logical data entities and their relationships within the system.

Entity	Relationship	Entity	Cardinality	Description
JOB	HAS	PIPELINE_STEP	1 : 7	Each job has exactly 7 steps
JOB	CONTAINS	INPUT_FILE	1 : Many	One job processes multiple uploaded files
JOB	PRODUCES	OUTPUT_FILE	1 : Many	One job generates multiple categorized CSVs
JOB	GENERATES	REPORT	1 : 1	Each job produces exactly one audit report
INPUT_FILE	YIELDS	RECORD	1 : Many	Each file contains multiple data records
RECORD	BELONGS_TO	CATEGORY	Many : 1	Records are grouped into categories
CATEGORY	MAPPED_TO	OUTPUT_FILE	1 : 1	Each category produces

Entity	Relationship	Entity	Cardinality	Description
				one output CSV

Key entity attributes:

- JOB: job_id (UUID), status, created_at, finished_at, error
- PIPELINE_STEP: step_name, step_number, status, log_output
- INPUT_FILE: original_name, saved_path, format (CSV/JSON), size_bytes
- RECORD: all data fields + _source tag for origin tracking
- OUTPUT_FILE: filename, category_name, record_count, download_url
- REPORT: duration_ms, total_read, duplicates_removed, clean_records, categories_generated

3. Project Description

3.1 Database

FlowCSV uses in-memory storage implemented using a JavaScript Map() object as its job store. This design choice was made deliberately for the following reasons:

- Simplicity: No external database dependency required for deployment.
- Speed: In-memory reads/writes are significantly faster than disk-based database queries for short-lived job state.
- Scope appropriateness: Pipeline jobs complete in under 30 seconds; persistent storage is not required for the current scope.

All processed files (outputs and archives) are stored on the local file system under structured directories organized by job ID. The file system acts as the persistent data store for actual data artifacts.

For a production deployment, the job store would be migrated to PostgreSQL or Redis without modifying any pipeline or API logic — the store.js module acts as an isolated data access layer.

3.2 Table Description

The following describes the logical structure of key data objects used in the system:

Job Object Schema

Field	Type	Description
id	String (UUID)	Unique identifier for each pipeline run
status	Enum String	Current state: queued | running | done | error
files	String[]	Array of original uploaded filenames
steps	StepObject[]	Array of 7 step objects with name, status, log
stats	Object	Runtime statistics: records read, dupes removed, categories
outputs	OutputObject[]	Array of generated files with name, count, download URL
report	Object	Final execution report with all metrics

Field	Type	Description
error	String | null	Error message if pipeline failed
createdAt	ISO DateTime	Timestamp when job was created
finishedAt	ISO DateTime	Timestamp when pipeline completed

Pipeline Step Object Schema

Field	Type	Description
name	String	Step name: Initialize, Read, Clean, Organize, Generate, Archive, Report
status	Enum String	pending | running | done | error
log	String	Detailed execution log output for this step

3.3 File / Database Design

The file system structure is organized as follows for each pipeline run:

Directory / File Path	Contents
server/uploads/	Temporary storage for uploaded files during processing
server/outputs/{job_id}/	All output files for a specific pipeline run
server/outputs/{job_id}/{category}.csv	One cleaned CSV file per detected category
server/outputs/{job_id}/_report.json	Execution audit report with full metrics
server/archives/{job_id}/	Archived copies of original uploaded source files

4. Input / Output Form Design

4.1 Input Design

The input interface is designed for simplicity and accessibility. No form fields, no configuration — just file upload.

Upload Page — Input Fields

Input Element	Type	Validation	Description
File Drop Zone	Drag & Drop / Click	.csv, .json only	Primary file input area accepting up to 20 files
File Size	Automatic check	Max 50MB per file	Rejected files shown with error message
Run Pipeline Button	Submit trigger	Min 1 file selected	Disabled until at least one valid file is added
Remove File (×)	Per-file action	N/A	Removes individual file from upload queue

Input Data Format — CSV

CSV files must have a header row. Column names are auto-detected. Recommended category-column names that enable automatic grouping: category, type, kind, group, tag, status, department.

Input Data Format — JSON

JSON files must contain either a single object or an array of objects at the root level. Nested objects are treated as single-value strings. Array-type JSON is the recommended format.

4.2 Output Design

Outputs are presented in the Job Dashboard view and available for immediate download.

Dashboard Output Elements

Output Element	Description
Progress Bar	Visual 0–100% completion bar updating in real time
Step Cards	One card per pipeline step showing status badge and execution log
Stats Panel	Records Read, Duplicates Removed, Clean Records, Categories — updated live
Output Files List	Downloadable links for each generated CSV file with row count
Execution Report	Duration, throughput (rec/sec), output file count, executed timestamp
History Page	List of all pipeline runs this session with status and summary metrics

Output File Format

Each output CSV file is named after its category value (e.g., engineering.csv, marketing.csv). It contains all columns from the original data except the internal _source tracking field added during processing. All records in a single output file belong to the same category value.

The _report.json file contains: jobId, executedAt, duration (ms), inputFiles[], stats (totalRecordsRead, duplicatesRemoved, cleanRecords, categoriesGenerated, outputFilesCreated), outputs[].

5. Testing & Tools Used

5.1 Testing Approach

FlowCSV was tested using a combination of manual functional testing, boundary testing, and edge case validation. The following test categories were performed:

Functional Testing

Test Case	Expected Result	Status
Upload single CSV file	Pipeline runs, 1 output file generated	PASS
Upload mixed CSV + JSON	Both formats parsed, combined output	PASS
Upload file with 200 duplicates	All 200 duplicates detected and removed	PASS
Upload file with dept. column	Auto-detected, grouped into dept. files	PASS
Upload file with no category column	Falls back to grouping by filename	PASS
Upload unsupported .xlsx file	Rejected at upload with error message	PASS
Download output CSV after pipeline	File downloaded correctly	PASS
View job in History page	Job appears with status and summary	PASS
Pipeline completes — polling stops	setInterval cleared on done/error	PASS

Boundary Testing

Boundary Condition	Behavior
0 files uploaded	Run Pipeline button disabled, no API call made
20 files uploaded (maximum)	All processed successfully
File with 0 data rows (header only)	Step 3 produces empty output, logged in report

Boundary Condition	Behavior
File with 5,000 rows	Processed in under 2 seconds, all records handled
All records are duplicates	0 clean records, empty output files, logged
Single record JSON file	Parsed as single-element array, processed normally

5.2 Tools Used for Testing

Tool	Purpose
Browser DevTools (Chrome)	Inspect network requests, console logs, React component state
Postman	API endpoint testing — /api/upload, /api/jobs/:id
Python (csv module)	Generated 5,000-row test dataset with 200 embedded duplicates
VS Code	Development, debugging, code review
Node.js console.log	Backend pipeline step-by-step logging during development

6. Implementation & Maintenance

6.1 Implementation Details

Phase 1 — Backend Foundation

The Express server was set up first with CORS, Multer file upload middleware, and basic route structure. The in-memory job store (store.js) was designed as an isolated module to allow future database replacement without touching pipeline logic.

Phase 2 — Pipeline Development

The 7-step processor was built step-by-step with each step independently testable. The async/await pattern was used throughout so the pipeline runs without blocking the Express event loop. Each step writes to the shared job object, enabling the polling mechanism to reflect live progress.

Phase 3 — Frontend Development

The React application was structured with three pages controlled by a central view state in App.js. react-dropzone was integrated for file handling. The polling mechanism using setInterval was implemented in JobPage.js with automatic cleanup on job completion.

Phase 4 — Integration & Testing

Both servers were run concurrently using the concurrently package. The React proxy configuration was set to route API calls from port 3000 to the Express server on port 3001. End-to-end testing was performed with real datasets including the 5,000-row employee test dataset.

6.2 Deployment Instructions

To run the application locally:

1. Install Node.js v18 or above from nodejs.org
2. Clone or extract the project to a local directory
3. Run: npm install (installs backend dependencies)
4. Run: cd client && npm install (installs frontend dependencies)
5. Terminal 1 — Backend: node server/index.js (runs on port 3001)
6. Terminal 2 — Frontend: cd client && npm start (runs on port 3000)
7. Open browser at http://localhost:3000

6.3 Maintenance Guidelines

Component	Maintenance Action
server/store.js	Replace Map() with PostgreSQL/Redis adapter for persistent storage
server/pipeline/processor.js	Add new step by inserting async function and updating step array
Category detection list	Add new field names to the sampleKeys.find() array in Step 4
File size / count limits	Modify Multer config in routes/upload.js (limits object)
Polling interval	Change POLL_INTERVAL constant in client/src/pages/JobPage.js
Output directory cleanup	Schedule periodic deletion of old job output folders using a cron job

7. Conclusion and Future Work

7.1 Conclusion

FlowCSV successfully demonstrates that a common, high-effort manual workflow — data file cleaning and categorical organization — can be fully automated through a well-structured full-stack application. The project achieves its stated objectives: a 4–6 hour manual process is reduced to under 30 seconds, duplicate records are eliminated with 100% accuracy, and outputs are organized, labeled, and ready for immediate use.

The 7-step pipeline architecture provides clear separation of concerns, making each processing stage independently auditable and maintainable. The real-time dashboard gives users visibility into the automation process rather than presenting it as a black box, building trust and transparency into the workflow.

From an academic perspective, the project reinforces the value of ETL (Extract, Transform, Load) pipeline thinking in software engineering — a paradigm used at scale in enterprise data systems, replicated here in an accessible, zero-configuration form for individual users and small teams.

7.2 Future Work

The following enhancements are planned for subsequent versions of FlowCSV:

Version 2 — Near-term Improvements

- User-selectable grouping column: After file upload and initial read, display detected column names in a dropdown allowing the user to choose the grouping field manually.
- Excel (.xlsx) support: Integrate the xlsx npm library to parse Microsoft Excel files alongside CSV and JSON.
- Persistent storage: Replace the in-memory job store with a PostgreSQL database so job history survives server restarts.
- Column mapping UI: Allow users to rename, merge, or exclude columns before the Generate step executes.

Version 3 — Scale Improvements

- Job queue with BullMQ: Handle concurrent pipeline runs from multiple users without resource contention.
- WebSocket-based updates: Replace 800ms polling with server-sent events or WebSockets for lower latency and reduced server load.
- Cloud deployment: Deploy backend on AWS EC2 or Railway, use Amazon S3 for file storage, serve frontend via CDN.

- User authentication: Add JWT-based login so teams share a deployment with separate job histories.

Version 4 — Intelligence Improvements

- ML-based column detection: Train a classifier on column name + value samples to identify grouping fields regardless of naming conventions.
- Fuzzy deduplication: Detect near-duplicates using edit distance algorithms (e.g., Levenshtein) to catch records differing only by whitespace or minor typos.
- Schema inference: Automatically detect and report column data types, flag mixed-type columns, and suggest data quality improvements.
- Natural language query interface: Allow users to filter or transform data using plain English instructions processed by an LLM API.

8. Outcome

8.1 Project Deployment

FlowCSV has been implemented as a fully functional local web application. The complete source code is organized into a deployable full-stack project with separate frontend (React, port 3000) and backend (Express, port 3001) components. The application has been tested end-to-end with a 5,000-row real-world employee dataset containing 200 embedded duplicate records, successfully processing, cleaning, and organizing the data into 10 category-based output files in under 2 seconds.

8.2 Potential for Copyright / Open Source Publication

The FlowCSV source code is being prepared for publication as an open-source repository on GitHub under the MIT License. Documentation, contribution guidelines, and a sample dataset will be included to enable other developers and researchers to deploy and extend the system. Repository publication is targeted for April, 2026 following final code review and documentation completion.

9. Bibliography

[1] *Node.js Foundation.* Node.js Documentation — File System and Streams. Available: https://nodejs.org/en/docs/

[2] *Facebook Inc.* React — A JavaScript Library for Building User Interfaces. Available: https://react.dev

[3] *Expressjs.com.* Express.js 4.x API Reference. Available: https://expressjs.com/en/api.html

[4] *Adaltas.* csv-parse — CSV Parsing for Node.js. Available: https://csv.js.org/parse/

[5] *Kindo, E. & Winand, M.* "Use The Index, Luke — A Guide to Database Performance for Developers." Available: https://use-the-index-luke.com

[6] *Multer npm package.* Multer — Node.js Middleware for Handling multipart/form-data. Available: https://www.npmjs.com/package/multer

[7] *Kimball, R. & Ross, M.* The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd Edition. Wiley, 2013.

[8] *MDN Web Docs.* JavaScript Reference — Set, JSON, Fetch API. Available: https://developer.mozilla.org

[9] *react-dropzone contributors.* react-dropzone Documentation. Available: https://react-dropzone.js.org

[10] *uuid npm package.* RFC 4122 UUID Generator for Node.js. Available: https://www.npmjs.com/package/uuid